DevOps philosophy and practice

Практическая презентация о культуре и управлении поставкой ПО

About Me

Aliaksandr Tsimokhau — Senior DevOps Engineer

- I've spent 10+ years in IT, but the last 6+ taught me that DevOps is an operating model, not a job title
- I work at the intersection of engineering, operations, and delivery, where feedback loops get shorter or the system breaks
- **Current Role:** Technical Support Lead at UFL Strikerz Inc
  - Support high-performance gaming platforms
  - Reduce friction between development and operations
  - Turn incidents, releases, and feedback into better workflows and fewer excuses

Agenda

1. Why DevOps exists

Что ломает старую модель доставки

2. Operating model

Культура, сотрудничество и четыре столпа

3. Anti-patterns

Антипаттерны разрозненных команд и культура обвинений

4. How DevOps ships

CI/CD · DevSecOps · метрики · toolchain

5. Continuous improvement

Первые шаги, измерение и постоянное улучшение

Part 1: Why DevOps exists

DevOps появился, чтобы каждое изменение доходило до пользователя быстрее и без потери качества

DevOps philosophy

Управление поставкой ПО через культуру и обратную связь

- **DevOps** соединяет разработку, эксплуатацию и бизнес-цели
- Главная цель - сократить расстояние между идеей и ценностью для пользователя
- Сильный DevOps строится на общих целях, общей ответственности и быстрой обратной связи
- Инструменты помогают, но сами по себе не создают культуру

Continuous journey

DevOps - это путь, а не конечное состояние

- Нет универсального шаблона внедрения
- DevOps не заканчивается после одной трансформации
- Разные команды и компании проходят этот путь по-разному
- Важно не "достичь DevOps", а научиться постоянно улучшать систему поставки

Why DevOps appeared

Почему этот подход стал необходим именно сейчас

- Продукты стали сложнее: микросервисы, облака, распределенные системы
- Скорость изменений выросла, а цена ошибок тоже выросла
- Ручные передачи между командами больше не масштабируются
- Нужны короткие циклы обратной связи и устойчивые процессы

Problem

Силосы, задержки, ручные передачи

Approach

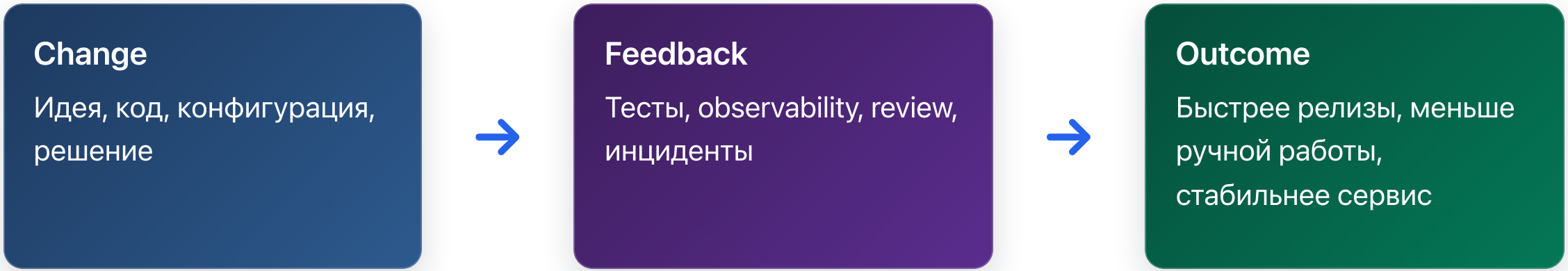
Общая ответственность, автоматизация, feedback loops

Result

Быстрее поставка, выше устойчивость, меньше трения

Flow system

DevOps как система потока изменений



Part 2: Operating model

Устойчивые изменения строятся на культуре, взаимодействии и инструментах

Four pillars

Operating model

Collaboration

Общая цель, общая ответственность, совместная работа

Proximity

Доверие, эмпатия, обмен контекстом и знаниями

Tools

Автоматизация, видимость, повторяемость, контроль

Scaling

Рост, зрелость, изменение структуры и процессов

Collaboration

Убирает стену между командами

- Общая цель важнее локальной оптимизации одной функции
- Dev и Ops должны видеть один и тот же результат, а не спорить о границах ответственности
- Внутри команды сначала нужна нормальная совместная работа, только потом - межкомандная
- Сотрудничество строится на общих ритуалах, прозрачности и взаимной ответственности

Metaphor

Оркестр работает не потому, что у каждого есть свой инструмент, а потому что есть общий ритм, партитура и дирижер

In DevOps

Команды играют по одной цели, видят общие сигналы и не ждут, пока кто-то "донесет партию" до следующего отдела

Proximity

Снижает стоимость передачи знаний

- Команды должны понимать контекст друг друга
- Чем меньше дистанция, тем легче доверие, эмпатия и обучение
- Общие цели не означают одинаковые KPI, но они требуют согласованных приоритетов
- Истории и примеры из практики помогают переносить знания между группами

Tools

Должны усиливать культуру, а не маскировать проблемы

- Инструмент выбирают под текущую организацию и ее цели
- Плохой инструмент увеличивает стоимость изменений
- Хорошие инструменты делают видимыми узкие места и дрейф процессов
- Автоматизация полезна только тогда, когда понятна логика процесса

Metaphor

Инструмент - это не двигатель изменений, а рычаг

Practical meaning

Если процесс сломан, новый инструмент просто ускорит поломку

Scaling

Проверяет, работает ли подход в жизни

- То, что работает в маленькой команде, может сломаться в большой организации
- При росте меняются процессы, границы и зоны ответственности
- Масштабирование касается не только роста, но и зрелости, и даже сокращения
- DevOps-практики должны переживать изменение размера команды и бизнеса

Part 3: Anti-patterns

Большинство провалов DevOps связаны не с технологиями, а с организацией

Myths

Типичные заблуждения

- DevOps - это не только разработчики и администраторы
- DevOps - это не отдельная "магическая" команда
- DevOps - это не сертификат и не должность
- DevOps - это не только для стартапов
- DevOps - это не просто автоматизация

Metaphor

Повесить табличку "DevOps" на одну комнату - все равно что назвать кассу магазином и ждать, что логистика, склад и доставка станут работать сами

Takeaway

Переименование команды не меняет поток ценности и не убирает разрозненность между командами

Anti-patterns

Похожи на трещины в плотине

Metaphor

Снаружи все может выглядеть нормально, но маленькие трещины в процессах постепенно превращаются в большой прорыв

In DevOps

Культура обвинений, обходные решения / костыли и героизм скрывают слабые места до первого серьезного инцидента

Culture killers

Антипаттерны, которые ломают культуру

- **Blame culture** — культура обвинений вместо обучения
- **Silos** — барьеры между командами и функциями
- **Heroics** — героизм вместо устойчивых процессов
- **Workarounds** — обходные решения / костыли вместо системного улучшения

Wrong question

"Когда мы уже внедрим DevOps?"

- DevOps нельзя считать разовой фазой проекта
- Это непрерывная работа с культурой, практиками и системой измерения
- Важнее траектория улучшения, чем формальный статус
- Ключевой показатель зрелости - способность учиться на изменениях

Part 4: How DevOps ships

Философия становится полезной, когда ее видно в поставке

Daily loop

DevOps в ежедневной работе

Plan

Согласовать цель и ограничить scope



Build

Собрать, протестировать, проверить



Run

Развернуть, наблюдать, улучшать

SDLC — software development lifecycle

Меняется, когда появляется DevOps

Traditional flow

- Requirements
- Design
- Implementation
- Testing
- Deployment
- Maintenance

With DevOps

- Short feedback loops
- Shared responsibility
- Automated tests and deploys
- Observability from day one
- Faster recovery

Core practices

Практики DevOps

Version control

Все код, конфиги и пайплайны в Git

CI

Автоматическая сборка и тестирование при каждом изменении

CD

Доставка изменений в тестовые и production-среды

IaC

Инфраструктура описана и воспроизводима как код

Observability

Метрики, логи и трейсы для быстрой обратной связи

GitOps

Git как источник истины для состояния инфраструктуры

CI/CD continuum

От integration к deployment



Delivery loop



Metrics

Четыре сигнала здоровья системы поставки

Lead time

Deployment frequency

Change failure rate

MTTR

DevSecOps

Делает безопасность частью пайплайна

Build

SAST, dependency scan, secrets check



Test

DAST, vulnerability scan, policy checks



Deploy + Run

Config validation, runtime monitoring, incident response

Tooling

Инструменты DevOps

CI/CD

GitLab CI, GitHub Actions, Jenkins

Containers

Docker, Podman, Kubernetes, Helm

IaC and config

Terraform, Ansible, Helmfile

Observability

Prometheus, Grafana, OpenTelemetry

Security

Trivy, Snyk, checkov, gitleaks

GitOps

Argo CD, Flux CD

Toolchain layers

1. Source and change control

Git, branching, review, traceability

2. Build and release automation

CI, packaging, artifact registry, versioning

3. Runtime platform

Containers, Kubernetes, Helm, configuration

4. Feedback and security

Metrics, logs, traces, scans, alerts

Part 5: Continuous improvement

Первые шаги важнее идеальной трансформации

Continuous improvement

Первые шаги важнее идеальной трансформации

1. Shared goal

Сначала договоритесь, что именно улучшаете

2. Highest-friction manual step

Уберите одно повторяющееся узкое место

3. Visible feedback

Сделайте результат измеримым и прозрачным



4. Collaboration

Сократите расстояние между людьми и командами

5. Right tools

Закрепите новый процесс, а не маскируйте старый

6. Repeat and measure

Улучшайте систему небольшими итерациями

Takeaways

- DevOps - это культура поставки ценности, а не набор инструментов
- Сильная трансформация держится на четырех столпах
- Ошибки чаще всего связаны с людьми и организацией, а не с YAML
- Лучший старт - маленькое, но системное улучшение с измеримым эффектом
- Работа DevOps - это CI/CD, безопасность, метрики и обратная связь в ежедневной работе

Culture

Общие цели и ответственность

Practice

Автоматизация и feedback loops

Scale

Процессы, которые выдерживают рост